DAY30-COMMAND-INJECTION

PART A — FUNDAMENTALS (answer in your own words):

1. What is OS Command Injection?

- OS Command Injection is a security vulnerability that allows attackers to inject operating-system commands through untrusted user input.
- It can allow attackers to execute unauthorized commands on the server.

2. What is an operating-system shell?

- A shell is a program that allows users to communicate with the operating system.
- It interprets and executes commands, such as whoami, ls and pwd.

3. Why is directly including user input inside shell commands dangerous?

- Attackers can manipulate the command using specially crafted input.
- This may cause the server to execute unauthorized operating-system commands.

4. What does whoami do?

- The whoami command displays the username of the currently logged-in operating-system user.
- It helps identify which account is executing a command.

5. What is server-side validation?

- Server-side validation checks user input on the server before processing it.
- It helps prevent invalid or malicious data from being accepted, even if browser-side validation is bypassed.

6. What is allowlist validation?

- Allowlist validation accepts only explicitly permitted input values or formats.
- For example, allowing only valid numeric store IDs and rejecting unexpected characters.

DAY30-COMMAND-INJECTION

7. Explain Least Privilege.

- Least Privilege means giving users and applications only the permissions necessary to perform their tasks. It limits damage if an application is compromised.

8. Why are safer platform APIs preferable to shell commands?

- Safer platform APIs perform operations directly without requiring shell command execution.
- They reduce the risk of attackers injecting additional commands through user input.

9. Explain SQL Injection vs OS Command Injection.

- SQL Injection: Attackers manipulate database queries through untrusted input to access or modify database information.
- OS Command Injection: Attackers inject operating-system commands through untrusted input to execute unauthorized commands on a server.

10. Explain XSS vs OS Command Injection.

- Cross-Site Scripting (XSS): Injected scripts execute in a victim's web browser and can affect users.
- OS Command Injection: Injected commands execute on the server's operating system and can affect the server.

PART B — PORTSWIGGER LAB (complete "OS command injection, simple case"):

1. Which functionality was vulnerable: Product stock checking

2. Which request parameters were present: productId & storeId

3. Which parameter was modified: storeId

4. What authorized training value was used: 1|whoami

5. Which harmless command executed: whoami

6. What does that command return: returns username

7. Record the OS username returned by YOUR lab instance: peter-PelrYB

8. Which Burp Suite feature was used?

Burp Suite Repeater, to modify and resend the HTTP request.

DAY30-COMMAND-INJECTION

9. Did PortSwigger show Solved: YES

10. Include one safe screenshot.



PART C — DEFENSIVE ANALYSIS:

Scenario: User Store ID → Raw String Concatenation → Shell Command → Execution

1. What is insecure about this design?

- User input is directly combined with a shell command without proper protection.
- Attackers can manipulate the input to execute unauthorized operating-system commands.

2. What safer architecture should be used?

- Avoid using shell commands whenever possible and use safer platform APIs instead.
- Keep user input separate from command execution and process validated values safely.

3. If Store IDs are only 1, 2, and 3, what validation should be applied?

- Use allowlist validation to accept only 1, 2, or 3.

4. Why should Least Privilege be used?

- The application should have only the minimum permissions required to perform its tasks.
- This limits the potential damage if an attacker successfully exploits the vulnerability.

5. How should internal command errors be handled?

- Internal command errors should be logged securely on the server for troubleshooting.
- Users should receive generic error messages without exposing sensitive system information.

DAY30-COMMAND-INJECTION

PART D — COMPARISON:

Compare SQL Injection, Cross-Site Scripting, OS Command Injection.

For each include: Interpreter, Target, Main security risk, Main defensive approach.

Category	SQL Injection (SQLi)	Cross-Site Scripting (XSS)	OS Command Injection
Interpreter	SQL database interpreter	Web browser (HTML/JavaScript interpreter)	Operating-system shell
Target	Database	Users' web browsers	Server operating system
Main security risk	Unauthorized access, modification or deletion of database information	Malicious scripts executing in users' browsers	Unauthorized execution of operating-system commands
Main defensive approach	Parameterized queries (prepared statements)	Context-aware output encoding	Avoid shell commands and use safer platform APIs

PART E — REFLECTION:

Write 5–8 sentences explaining USER DATA vs OPERATING-SYSTEM COMMANDS.

User Data	Operating-System Commands
User data is information supplied by users, such as a Store ID / search term.	Operating-system commands are instructions used to perform tasks on a computer.
It should be treated as untrusted input.	They are interpreted and executed by the operating system through a shell or command processor.
User data should be processed as information, not as instructions.	Commands can perform actions such as displaying files or identifying the current user.
Directly combining user data with shell commands can be dangerous.	Unsafe command execution may allow attackers to run unauthorized commands.
Input validation helps ensure user data follows expected formats.	Safer platform APIs help avoid unnecessary shell execution.

DAY30-COMMAND-INJECTION

User data should never be allowed to change the intended command.

Commands should execute only with the permissions necessary to perform their tasks.